

CREDIT CARD MANAGEMENT SYSTEM

Java Full Stack Training – Case Study Project Abstract

1. Project Overview

The Credit Card Management System is a web-based application designed to manage the complete lifecycle of credit cards and their related customer activities. The system allows bank or financial-institution users to maintain customer information, issue and manage credit cards, monitor credit limits and available balances, record transactions, process payments, and generate useful reports.

This case study is intended for trainees who have completed Java Full Stack training. The objective is to apply Java, Spring Boot, REST APIs, JPA/Hibernate, database concepts, frontend development, validation, exception handling, testing, and basic security concepts in one integrated real-world-style application.

2. Business Problem

Managing credit card information manually or through disconnected systems can result in duplicate data, difficulty tracking transactions and payments, and limited visibility into customer credit-card status. The proposed system provides a centralized application for managing customers, cards, transactions, payments, and credit information.

3. Project Objectives

- Maintain customer and credit-card information in a centralized system.
- Provide functionality for credit-card issuance, activation, blocking, and status management.
- Track credit limits, outstanding amounts, available credit, and payment information.
- Record and view credit-card transactions.
- Allow authorized users to record customer payments and update outstanding balances.
- Provide search, filtering, and basic reporting functionality.
- Implement validation and meaningful error handling for invalid business operations.
- Develop REST APIs and integrate them with a responsive web-based frontend.
- Apply unit testing and integration testing to important application components.
- Demonstrate clean coding practices and suitable object-oriented/design principles.

4. Major Functional Areas

Customer Management

- Create, view, update, and search customer details.
- Maintain customer contact information and customer status.
- View cards associated with a customer.

Credit Card Management

- Issue a new credit card to an eligible customer.
- Maintain card number/reference, card type, credit limit, available credit, expiry date, and status.
- Activate, block, unblock, or close a card based on business rules.
- Display card details without exposing sensitive information unnecessarily.

Transaction Management

- Record purchase/debit transactions against a credit card.
- View transaction history for a card or customer.
- Support transaction categories such as purchase, cash withdrawal, fee, refund, and adjustment.
- Validate transactions against available credit and card status.

Payment Management

- Record payments made against a credit card.
- Update outstanding balance and available credit after a successful payment.
- Maintain payment date, amount, mode, and reference details.
- Allow users to view payment history.

Dashboard and Reports

- Display customer/card summary information.
- Show card status, available credit, outstanding amount, and recent transactions.
- Provide basic reports such as active cards, blocked cards, outstanding balances, and transaction summaries.

User Access

- Provide login functionality for application users.
- Implement role-based access where appropriate, such as ADMIN and CUSTOMER-SERVICE roles.
- Restrict selected operations based on user role.

5. Suggested Technology Stack

- Backend: Java, Spring Boot, Spring MVC / REST, Spring Data JPA, Hibernate.
- Database: MySQL (preferred) or another relational database.
- Frontend: React JS or Angular.
- Build Tool: Maven.
- Testing: JUnit 5 and Mockito.
- API Testing: Postman or equivalent REST client.
- Version Control: Git/GitHub.
- Development Environment: IntelliJ IDEA, Eclipse, or Spring Tool Suite.
- Optional: Spring Security and JWT for authentication and authorization.

6. High-Level Entities

Trainees should identify the required entities and relationships during the design phase. The following entities are suggested:

- Customer – customer identification and contact details.
- CreditCard – card information, credit limit, available credit, status, and dates.
- Transaction – transaction details, amount, date/time, type, and card reference.
- Payment – payment amount, date, mode, reference, and card reference.
- User – application login and role information.
- Optional supporting entities: CardType, Address, Merchant, AuditLog, or Statement.

7. Important Business Rules

- A credit card must belong to a valid customer.
- Only an active card can normally be used for purchase transactions.
- A transaction amount must be positive.
- A purchase transaction must not exceed the available credit.
- A payment cannot be greater than the applicable outstanding amount unless the business design explicitly supports overpayment.
- Blocking or closing a card should prevent new purchase transactions.
- Card numbers and other sensitive information should not be exposed unnecessarily in API responses or the user interface.
- All important operations should have appropriate validation and error handling.
- Transaction and payment operations should maintain consistent account/card balances.

8. Expected REST API Areas

Trainees should design appropriate REST endpoints. Example resource areas include:

- POST /api/customers
- GET /api/customers/{id}
- GET /api/customers
- POST /api/cards
- GET /api/cards/{id}
- PUT /api/cards/{id}/activate
- PUT /api/cards/{id}/block
- POST /api/cards/{id}/transactions
- GET /api/cards/{id}/transactions
- POST /api/cards/{id}/payments
- GET /api/cards/{id}/payments
- GET /api/reports/card-summary

9. Non-Functional Requirements

- The application should follow a layered architecture such as Controller, Service, Repository, and Entity/DTO layers.
- Use appropriate validation, exception handling, logging, and HTTP status codes.
- Use DTOs where appropriate rather than exposing persistence entities directly.
- Use meaningful naming conventions and maintainable code structure.
- Avoid duplicate business logic.
- Write unit tests for service/business logic and controller tests where appropriate.
- The application should be designed so that additional features can be added with minimal changes to existing functionality.
- Database operations should use transactions where required to maintain data consistency.

10. Expected Trainee Deliverables

- Requirements understanding and assumptions document.
- Use-case diagram and/or functional flow diagram.
- ER diagram and database design.
- UML class diagram and relevant sequence/activity diagrams.
- Database schema and SQL scripts.
- Complete Spring Boot backend application.
- Complete frontend application integrated with REST APIs.
- JUnit/Mockito test cases for important functionality.
- Postman collection or API documentation.
- README containing setup instructions, database configuration, application execution steps, and sample credentials if authentication is implemented.

11. Suggested Development Phases

Phase 1 – Analysis: Identify actors, requirements, use cases, assumptions, and business rules.

Phase 2 – Design: Prepare database design, UML diagrams, API design, and application architecture.

Phase 3 – Backend Development: Develop entities, repositories, services, controllers, DTOs, validation, and exception handling.

Phase 4 – Frontend Development: Develop screens, forms, navigation, validations, API integration, and user-friendly views.

Phase 5 – Testing: Develop unit, integration, and API tests and fix identified defects.

Phase 6 – Integration and Deployment: Integrate frontend and backend, configure the database, prepare deployment documentation, and demonstrate the application.

12. Suggested Screens

- Login page
- Dashboard
- Customer registration and customer search
- Customer details page
- Credit-card issuance page
- Credit-card details/status management page
- Transaction entry and transaction history page
- Payment entry and payment history page
- Reports/summary page

13. Case Study Expectations

Trainees are expected to make reasonable design decisions rather than implementing every detail exactly as prescribed. They should document assumptions and demonstrate how the frontend, REST APIs, business layer, persistence layer, and database work together.

The case study may be implemented individually or by a small team. The trainer may introduce additional business rules, validation scenarios, or reporting requirements during the implementation and review sessions.

14. Evaluation Areas

- Requirement analysis and completeness.
- Object-oriented design and appropriate use of Java concepts.
- Spring Boot architecture and REST API design.
- Database design and JPA/Hibernate implementation.
- Frontend usability and API integration.
- Validation, exception handling, and security considerations.
- Unit and integration test coverage.
- Code quality, readability, maintainability, and avoidance of duplication.
- UML/design documentation.
- Overall working demonstration and explanation of technical decisions.

15. Final Project Outcome

At the end of the case study, trainees should be able to demonstrate a complete Java Full Stack application that manages the major activities associated with credit cards. The project should demonstrate their ability to move from requirements analysis and system design through backend development, database integration, frontend development, testing, and deployment readiness.